

claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_4fd1a4ea-6c0\agent-a8d1288a98d879c93.jsonl`  
- SHA-256: `1bd9e151bd78858932a770ba7a420dcdf937c3445d5a1564c38ff8c16f065468`  
- Source modified: `2026-07-02T09:30:45+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_4fd1a4ea-6c0`  
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

Transcript

1. user (2026-07-02T09:20:08.328Z)

You are reviewing a god-module extraction for SEAM FIDELITY.
REFACTOR UNDER REVIEW (uncommitted, in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs, branch refactor/p23-orchestration-split off origin/master):

- NEW backend/orchestration.py (739 lines): 10 members moved VERBATIM from backend/main.py: `_finalize_reingest`, `_report_config_for_input`, `_ingest_remote_segments`, `_cloud_available`, `_maybe_dispatch_remote`, `_execute_processing`, `_remote_input_size_bytes`, `_CompileError`, `_resolve_compile`, `_execute_compile`.
- backend/main.py 2360 -> 1703 lines; re-exports all 10 names via "from backend.orchestration import (...)" (noqa F401);
5 now-unused imports removed (csv, math, Tuple, StitchingConfig, localize_watermark).
- Enumerated intentional deltas (everything else is AST-identical, machine-verified):
(1) `_maybe_dispatch_remote`: "req: ProcessRequest" -> string forward-ref "ProcessRequest" (TYPE_CHECKING import).
(2) `_execute_processing`: inserted "import backend.main as `_main`" after docstring; `global_config/db_instance/ProcessRequest` resolved via `_main.<name>` at call time (the `job_worker` seam convention).
(3) two "# type: ignore[...]" comments (AST-invisible).
- NEW tests/test_main_module_budget.py: line budget 1750, route ceiling 18, re-export identity check.
- Verified already: AST-identity (10 moved + 51 retained members), ruff+format clean, mypy green (142 files),
ratchet test passes. Full suite running separately.

Inspect with: `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" diff` (and `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" status`); the new files are untracked/modified in the worktree.
READ-ONLY: do not modify the worktree, do not checkout/commit.

Hunt for broken seams:

1. Enumerate EVERY monkeypatch.setattr(m|backend.main, "<name>", ...) / patch("backend.main.<name>") in tests/ and check: after the move, does patching `backend.main.<name>` still affect the code path that consumes it? (Key risk: a moved function calling another moved function BARE ? patching main's re-export then does NOT intercept the internal call. Master behavior: all lookups went through main's module dict, so patching main DID intercept. Find any test that patches one of the 10 moved names AND exercises a code path where another MOVED function calls it internally bare.)
2. backend/api/job_worker.py resolves `_main._execute_processing/_execute_compile` at call time ? confirm re-export covers it.
3. backend/main.py retained code (routes, CLI main()) references moved names bare (`_resolve_compile`, `_CompileError`, `_finalize_reingest`, `_report_config_for_input`, `_remote_input_size_bytes`) ? confirm each resolves via the re-export and that patching

backend.main.<name> still intercepts those ROUTE call sites.
4. tests calling m.<name> directly or importing from backend.main.
Report ONLY real, evidence-backed findings (file:line).

2. user (2026-07-02T09:20:16.605Z)

M backend/main.py
?? backend/orchestration.py
?? tests/test_main_module_budget.py
30736ab docs: make AUDIT_REPORT the umbrella code-audit doc; fold + archive the rest (#430)
65983ac docs: reconcile audit docs to shipped state + record PR dispositions (#429)

3. user (2026-07-02T09:20:18.494Z)

Found 8 files
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_main_module_budget.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_api_cloud_dispatch.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_job_worker_module_seam.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_base.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_async_jobs.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_api_endpoints.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_api_annotations.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\tests\test_api_assets.py

4. user (2026-07-02T09:20:32.291Z)

```
tests/test_api_annotations.py:91: monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a, **k: len(payload))
tests/test_api_assets.py:129: monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a, **k: len(payload))
tests/test_api_endpoints.py:815: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: len(payload))
tests/test_api_endpoints.py:851: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10 * 1024**3)
tests/test_api_endpoints.py:868: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: None)
tests/test_async_jobs.py:503: mp.setattr(m, "_execute_compile", _fake_compile)
```

5. user (2026-07-02T09:20:44.269Z)

```
3:The full hash -> validate -> segment -> report pipeline (`_execute_processing`), the
4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` / `_ingest_remote_segments`),
5:and the shared compile resolution + worker handler (`_resolve_compile` /
`_execute_compile`).
13:...)` and `import backend.main as m; m._execute_processing(...)` keep working unchanged.
45:def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
58:def _report_config_for_input(config: Any, raw_input: str) -> Any:
86:def _ingest_remote_segments(
136:def _cloud_available(cfg) -> bool:
155:def _maybe_dispatch_remote(
204:    # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
215:    if not _cloud_available(cfg):
298:        _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
306:    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops
```

the old ones.

```
310:    _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
342: def _execute_processing(
350:     test code that calls ``_execute_processing(req)`` still works.
357:     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
360:     # Old signature: _execute_processing(req, progress=None)
411:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
497:     remote = _maybe_dispatch_remote(
534:     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
578:     config=_report_config_for_input(config, req.video_path),
593: def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
614: class _CompileError(Exception):
624: def _resolve_compile(
631:     ``_CompileError(404/400)``. Returns ``(video_row, kept_segments, safe_out_name)``. """
634:     raise _CompileError(404, "Indexed video record not found.")
639:     raise _CompileError(400, str(e)) from e
644:     raise _CompileError(400, "No matching play segments found to stitch.")
665:     raise _CompileError(
673: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
686:     video, kept, out_name = _resolve_compile(
689:     except _CompileError as e:
```

6. user (2026-07-02T09:20:46.055Z)

```
303: # backend.main ? `import backend.main as m; m._execute_processing(...)` , the job worker's
304: # call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main, ...)
307:     _cloud_available,
308:     _CompileError,
309:     _execute_compile,
310:     _execute_processing,
311:     _finalize_reingest,
312:     _ingest_remote_segments,
313:     _maybe_dispatch_remote,
314:     _remote_input_size_bytes,
315:     _report_config_for_input,
316:     _resolve_compile,
338:     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
473:     "cloud_available": _cloud_available(cfg),
643:     need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
872:     need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
1017:     need = _remote_input_size_bytes(
1170:     video, _kept, out_name = _resolve_compile(
1177:     except _CompileError as e:
1618:         _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1628:         config=_report_config_for_input(global_config, args.video_path),
```

7. user (2026-07-02T09:21:08.651Z)

```
1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5     and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8     SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
```

```

through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     """
16
17     import csv
18     import json
19     import logging
20     import math
21     import os
22     import tempfile
23     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
24     from urllib.parse import quote
25
26     from backend import storage
27     from backend.config import AppConfig, StitchingConfig
28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.stitcher.compiler import VideoCompiler
30     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
31     from backend.pipeline.validators.base import registry
32     from backend.reporting import emit_indexer_report
33     from backend.results import Failure
34     from backend.utils.hashing import compute_video_id
35     from backend.utils.paths import output_base, safe_output_filename
36
37     if (
38         TYPE_CHECKING
39     ): # runtime import would be circular (main imports us for the re-export)
40         from backend.main import ProcessRequest # noqa: F401
41
42     logger = logging.getLogger(__name__)
43
44
45     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
46         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
47
48         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
49         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
50         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
51         """
52         if segments:
53             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
54         else:
55             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
56
57
58     def _report_config_for_input(config: Any, raw_input: str) -> Any:
59         """Return report config that keeps remote-input reports out of disposable
scratch."""
60         storage_config = (
61             config.get("storage")
62             if isinstance(config, dict)
63             else getattr(config, "storage", None)
64         )
65         try:
66             if storage.input_is_local(raw_input, storage_config):
67                 return config

```

```

68     except ValueError:
69         return config
70     try:
71         cfg = (
72             config.model_dump(mode="json")
73             if hasattr(config, "model_dump")
74             else dict(config or {})
75         )
76     except Exception:
77         cfg = {}
78     report = dict(cfg.get("report") or {})
79     if report.get("output_dir"):
80         return config
81     report["output_dir"] = os.path.join(output_base(config), "reports")
82     cfg["report"] = report
83     return cfg
84
85
86     def _ingest_remote_segments(
87         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
88     ) -> int:
89         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
90         the bucket and
91         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
92         call ? so
93         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
94         were produced.
95         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
96         pattern (cloud
97         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
98         malformed row is
99         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
100         temp dir is
101         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
102         a job failure
103         and prunes any partial rows (destroy-AFTER-confirm)."""
104         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
105         ref = storage.parse_uri(uri)
106         n = 0
107         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
108             local = storage.fetch(
109                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
110             )
111             with open(local, newline="", encoding="utf-8") as f:
112                 for row in csv.DictReader(f):
113                     if n >= max_rows:
114                         logger.warning(
115                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
116                             rest.",
117                             max_rows,
118                             uri,
119                         )
120                     break
121                 try:
122                     start, end = float(row["start"]), float(row["end"])
123                 except (KeyError, TypeError, ValueError):
124                     continue
125                 if not (math.isfinite(start) and math.isfinite(end)):
126                     continue # reject inf/nan times rather than persist a junk rally
127                 meta: Dict[str, Any] = {
128                     "source": "cloud_run",
129                     "ending_reason": (row.get("ending_reason") or "").strip(),
130                 }
131                 sc = (row.get("shot_count") or "").strip()
132                 if sc:

```

```

125         try:
126             meta["shot_count"] = int(
127                 float(sc)
128             ) # OverflowError on 'inf'/'1e400'
129         except (ValueError, OverflowError):
130             meta["shot_count"] = sc
131         db.add_play_segment(video_id, start, end, metadata=meta)
132         n += 1
133     return n
134
135
136 def _cloud_available(cfg) -> bool:
137     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
138 3).
139     True when the control plane is wired for Cloud Run: either the configured default
140 target is
141 already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
142 coordinates
143 (``storage.output_root``)
144 are present so a forced per-request override can actually dispatch + collect a
145 result.
146 Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
147 would work."""
148 deployment = getattr(cfg, "deployment", None)
149 if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
150     return True
151 has_job = bool(
152     os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
153 )
154 out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
155 return bool(has_job and out_root)
156
157
158 def _maybe_dispatch_remote(
159     cfg,
160     db,
161     req: "ProcessRequest",
162     video_id: str,
163     seg_name: str,
164     val_results,
165     play_wm: int,
166     cand_wm: int,
167     notify,
168 ) -> Optional[Tuple[Dict[str, Any], bool]]:
169     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
170 ``deployment.compute_target
171 == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
172 intervals into
173 THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
174 Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
175 the cloud job
176 actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
177 observability report
178 doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
179 process segmenter
180 (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
181 non-``cloud_run`` target, so
182 the in-process path stays byte-identical)."""
183 from backend.compute import ComputeJobSpec, get_compute_backend
184
185 def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:

```

```

178         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
179         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
180         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
181         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
182         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
183         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
184         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
185         return (
186             {
187                 "video_id": video_id,
188                 "status": "failed",
189                 "message": msg,
190                 "results": [r.model_dump() for r in val_results],
191                 "play_segments": [],
192                 "compute": {
193                     "path": "cloud_run" if ran else "not_run",
194                     "requested": req.compute,
195                     "target": "cloud_run",
196                     "dispatched": ran,
197                 },
198             },
199             ran,
200         )
201
202         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
203         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
204         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
205         choice = (req.compute or "").strip().lower()
206         if choice and choice not in ("local", "cloud"):
207             logger.warning(
208                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
209                 req.compute,
210             )
211             choice = ""
212         if choice == "local":
213             return None # explicit "run on my machine" ? in-process regardless of the
server's target
214         if choice == "cloud":
215             if not _cloud_available(cfg):
216                 return _failed(
217                     "cloud-serving was requested but this server isn't configured for it "
218                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
219                     False,
220                 )
221             compute = get_compute_backend(cfg, target_override="cloud_run")
222         else:
223             compute = get_compute_backend(
224                 cfg
225             ) # server default (default-OFF unless cloud_run)
226         if compute is None:
227             return None # default-OFF ? run the in-process segmenter below (unchanged)
228
229         storage_cfg = getattr(cfg, "storage", None)
230
231         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local

```

```

to this box.
232     try:
233         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
234     except ValueError as e:
235         return _failed(
236             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
237         )
238     if input_ref.backend == "local":
239         return _failed(
240             "cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
241             "can't be read by the Cloud Run job. Put the video in your bucket first.",
242             False,
243         )
244     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
245     if not out_root:
246         return _failed(
247             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
248             "? that's where the Cloud Run job writes the result.",
249             False,
250         )
251
252     notify("dispatching (cloud_run)")
253     logger.info(
254         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
255         req.video_path,
256         out_root,
257     )
258     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
259     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
260     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
261     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
262     overrides = []
263     sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
264     if sk is not None:
265         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
266     spec = ComputeJobSpec(
267         video_uri=req.video_path,
268         out_uri=out_root,
269         segmenter=seg_name,
270         config_overrides=tuple(overrides),
271     )
272     try:
273         result = compute.run_infer(
274             spec
275         ) # dispatch the Cloud Run Job + bucket-poll to completion
276     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
277         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
278         return _failed(f"cloud dispatch error: {e}", True)
279     if result.returncode != 0:
280         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
281     if result.video_id and result.video_id != video_id:
282         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
283         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
284         logger.warning(
285             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "

```

```

286         "and the worker hash; ingesting under the local id.",
287         result.video_id,
288         video_id,
289     )
290
291     notify("ingesting (cloud_run)")
292     storage_dict = (
293         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
294         if hasattr(storage_cfg, "model_dump")
295         else {}
296     )
297     try:
298         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
299     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
300         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
301         return _failed(
302             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
303             True,
304         )
305
306     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
307     segments = [
308         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
309     ]
310     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
311     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
312     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
313     provenance = {
314         "path": "cloud_run",
315         "requested": req.compute,
316         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
317         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
318         "execution": result.execution,
319         "outputs_uri": result.outputs_uri,
320     }
321     logger.info(
322         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
323         result.execution,
324         provenance["job"],
325         provenance["region"],
326         result.outputs_uri,
327         video_id,
328     )
329     return (
330         {
331             "video_id": video_id,
332             "status": "success",
333             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
334             "results": [r.model_dump() for r in val_results],
335             "play_segments": segments,
336             "compute": provenance,
337         },
338         True,
339     )
340
341
342     def _execute_processing(
343         config=None, db=None, req=None, progress=None

```

```

344 ) -> Dict[str, Any]:
345     """The full hash ? validate ? segment ? report pipeline for one video.
346
347     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
348     module-level ``global_config``/``db_instance`` (backward-compatible with existing
349     tests that pass only ``req``). Route handlers always pass explicit params; older
350     test code that calls ``_execute_processing(req)`` still works.
351
352     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
353     Raises on unexpected internal errors ? the caller records those as a job failure
354     """
355     import backend.main as _main # call-time seam: main's globals/model stay patchable
356
357     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
358     # shift positional args ? req arrives in position 0 (config), progress in position 1
359     (db).
359     if config is not None and not isinstance(config, AppConfig):
360         # Old signature: _execute_processing(req, progress=None)
361         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
362             req = db
363             db = None
364         elif req is None and isinstance(config, _main.ProcessRequest):
365             req = config
366             config = None
367     # Resolve config/db from module globals when not passed explicitly
368     if config is None:
369         config = _main.global_config
370     if db is None:
371         db = _main.db_instance
372
373     notify = progress or (lambda stage: None)
374
375     notify("hashing")
376     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
377     (identity ?
original
378     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
379     # consumers (hash / validators / segmenter) use the resolved local path.
380     local_video = ""
381     try:
382         local_video = storage.acquire_local_input(
383             req.video_path, getattr(config, "storage", None)
384         )
385         video_id = compute_video_id(local_video)
386     except Exception:
387         storage.cleanup_acquired_local_input(
388             req.video_path, local_video, getattr(config, "storage", None)
389         )
390         raise
391     try:
392         existing_video = db.get_video(video_id)
393     except Exception:
394         storage.cleanup_acquired_local_input(
395             req.video_path, local_video, getattr(config, "storage", None)
396         )
397         raise
398     was_reingest = existing_video is not None
399
400     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
401     default_seg = getattr(
402         getattr(config, "indexing", None), "default_segmenter", "motion"
403 )

```

```

404     seg_name = req.segmenter or default_seg
405     segmenter_actual = None
406     segmentation_ran = False
407     val_results: List[Any] = []
408     val_context: Dict[str, Any] = {}
409     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
410     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
411     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
412     compute_actual: Optional[str] = None
413     response: Optional[Dict[str, Any]] = None
414     try:
415         if (
416             existing_video
417             and existing_video.get("status") == "processed"
418             and not req.force
419         ):
420             cached_segments = db.query_play_segments(video_id, {})
421             if cached_segments:
422                 logger.info(
423                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
424                     video_id,
425                 )
426                 response = {
427                     "video_id": video_id,
428                     "status": "success",
429                     "message": (
430                         f"Reused {len(cached_segments)} cached play segments. "
431                         "Pass force=true to reprocess this video."
432                     ),
433                     "results": existing_video.get("validation_results", []),
434                     "play_segments": cached_segments,
435                     "cached": True,
436                     "compute": {
437                         "path": "cached",
438                         "requested": getattr(req, "compute", None),
439                     },
440                 }
441                 return response
442
443     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
444     notify("validating")
445     logger.info(f"Running validations for {req.video_path}...")
446     val_results, val_context = registry.run_all_with_context(local_video, config)
447     failures = [r for r in val_results if not r.passed]
448
449     if failures:
450         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
451         # status; it no longer destroys the video's prior good segments.
452         db.add_video(
453             video_id,
454             req.video_path,
455             "failed",
456             [r.model_dump() for r in val_results],
457         )
458         response = {
459             "video_id": video_id,
460             "status": "failed",
461             "message": "Video failed validation checks.",
462             "results": [r.model_dump() for r in val_results],
463         }

```

```

464         return response
465
466     # 2. Run segmenter & indexer
467     logger.info("[API] Video passed checks. Segmenting and indexing...")
468     db.add_video(
469         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
470     )
471
472     segmenter_cls = segmenter_registry.get(seg_name)
473     if not segmenter_cls:
474         # Submit-time validation already 400s unknown names; this is the defensive
475         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
476         available = list(segmenter_registry.list_available().keys())
477         response = {
478             "video_id": video_id,
479             "status": "failed",
480             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
481             "results": [r.model_dump() for r in val_results],
482             "play_segments": [],
483         }
484         return response
485
486     logger.info(f"[API] Using segmenter: {seg_name}")
487     notify(f"segmenting ({seg_name})")
488     segmenter_actual = seg_name
489     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
490     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
491     play_wm, cand_wm = db.segment_watermarks(video_id)
492     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
493     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
494     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
495     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
496     # they're dropped on the cloud path in this slice ? a documented follow-up.)
497     remote = _maybe_dispatch_remote(
498         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
499     )
500     if remote is not None:
501         response, segmentation_ran = (
502             remote # `ran` = did the cloud job actually execute (report)
503         )
504         return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
505     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
506     compute_actual = "in_process"
507     segmenter = segmenter_cls(db, config)
508     try:
509         result = segmenter.process_video(
510             video_id, local_video, req.player_pool, req.max_frames
511         )
512     except Exception:
513         # A raising segmenter must not leave half-written rows: restore the pre-run
514         # state (same destroy-after-confirm contract as the Failure branch), then
let
515         # the job worker record the failure.
516         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
517         raise

```

```

518         segmentation_ran = True
519
520     if isinstance(result, Failure):
521         logger.error(f"[API] Segmenter failed: {result.message}")
522         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
523         response = {
524             "video_id": video_id,
525             "status": "failed",
526             "message": f"Segmenter '{seg_name}' failed: {result.message}",
527             "results": [r.model_dump() for r in val_results],
528             "play_segments": [],
529         }
530         return response
531
532     segments = result.value if hasattr(result, "value") else result
533     notify("finalizing")
534     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
535
536     response = {
537         "video_id": video_id,
538         "status": "success",
539         "message": f"Successfully indexed {len(segments)} play segments using
540 '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
541         "results": [r.model_dump() for r in val_results],
542         "play_segments": segments,
543     }
544     return response
545
546     finally:
547         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
548         already set
549         # response["compute"] (path=cloud_run + the Cloud Run execution); for every
550         other path stamp
551         # the actual path here so the job result PROVES which backend ran (in_process /
552         not_run) and
553         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
554         then visible.
555         if isinstance(response, dict):
556             response.setdefault(
557                 "compute",
558                 {
559                     "path": compute_actual or "not_run",
560                     "requested": getattr(req, "compute", None),
561                 },
562             )
563         prov = response.get("compute", {}) if isinstance(response, dict) else {}
564         logger.info(
565             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
566             video_id,
567             getattr(req, "compute", None),
568             prov.get("path"),
569             prov.get("dispatched"),
570             (response or {}).get("status") if isinstance(response, dict) else None,
571         )
572         # Always emit the per-video observability report (next to the video, or to
573         # report.output_dir). Never raises. Surface the paths in the response. (The
574         compute-path
575         # proof lives on the job result + the [COMPUTE] log, not the report ?
576         obsreport's record_run
577         # doesn't surface arbitrary run signals.)
578         paths = emit_indexer_report(
579             db=db,
580             video_id=video_id,
581             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
582             the

```

```

574         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
575         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
576         # still preserved in DB provenance via db.add_video.
577         video_path=local_video,
578         config=_report_config_for_input(config, req.video_path),
579         val_results=val_results,
580         val_context=val_context,
581         segmenter_requested=req.segmenter,
582         segmenter_actual=segmenter_actual,
583         was_reingest=was_reingest,
584         segmentation_ran=segmentation_ran,
585     )
586     if paths and isinstance(response, dict):
587         response["reports"] = paths
588     storage.cleanup_acquired_local_input(
589         req.video_path, local_video, getattr(config, "storage", None)
590     )
591
592
593     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
594         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
595
596         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
597         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
598         to a no-op rather than blocking a job it cannot measure."""
599         try:
600             sc = (
601                 storage_config.model_dump()
602                 if hasattr(storage_config, "model_dump")
603                 else (storage_config or {})
604             )
605             backend = storage.get_storage(input_ref.backend, config=sc)
606             return int(backend.stat(input_ref).size)
607         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
608             logger.debug(
609                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
610             )
611             return None
612
613
614     class _CompileError(Exception):
615         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
616         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
617
618         def __init__(self, status_code: int, detail: str):
619             super().__init__(detail)
620             self.status_code = status_code
621             self.detail = detail
622
623
624     def _resolve_compile(
625         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
626     ):
627         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
628         row could change between submit and run). Verifies the video exists, the requested
segments match
629         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known

```

```

shot_count are exempt
630         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
631         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``. """
632         video = db.get_video(video_id)
633         if not video:
634             raise _CompileError(404, "Indexed video record not found.")
635         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
636         try:
637             out_name = safe_output_filename(output_filename)
638         except ValueError as e:
639             raise _CompileError(400, str(e)) from e
640
641         all_segments = db.query_play_segments(video_id, {})
642         matched = [s for s in all_segments if s["id"] in segment_ids]
643         if not matched:
644             raise _CompileError(400, "No matching play segments found to stitch.")
645
646         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
647         kept = []
648         for s in matched:
649             meta = s.get("metadata") or {}
650             sc = meta.get("shot_count") if isinstance(meta, dict) else None
651             try:
652                 if sc is not None and int(sc) < stitching.min_shot_count:
653                     continue
654             except (TypeError, ValueError):
655                 pass # unparseable count -> treat as unknown, keep
656             kept.append(s)
657         if len(kept) < len(matched):
658             logger.info(
659                 "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
660                 stitching.min_shot_count,
661                 len(matched) - len(kept),
662                 len(matched),
663             )
664         if not kept:
665             raise _CompileError(
666                 400,
667                 f"All {len(matched)} matching segments fall below "
668                 f"stitching.min_shot_count={stitching.min_shot_count}.",
669             )
670         return video, kept, out_name
671
672
673     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
674         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
675         and return the {status, filepath, download_url} result the SPA polls for. Returns a
676         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
677         so the SPA always gets a clean verdict instead of a worker-crash error."""
678         notify = progress or (lambda stage: None)
679         params = json.loads(job.get("params") or "{}")
680         video_id = job.get("video_id") or ""
681         segment_ids = params.get("segment_ids") or []
682         locale = params.get("locale")
683
684         notify("resolving")
685         try:
686             video, kept, out_name = _resolve_compile(
687                 db, cfg, video_id, segment_ids, params.get("output_filename", "")

```

```

688         )
689     except _CompileError as e:
690         return {"status": "failed", "message": e.detail, "video_id": video_id}
691
692     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
693     output_dir = (
694         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
695     )
696     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
697     # unknown/absent locale keeps the configured default unchanged.
698     localized_watermark = localize_watermark(stitching.watermark, locale)
699     compiler = VideoCompiler(
700         output_dir,
701         begin_padding=stitching.begin_padding,
702         end_padding=stitching.end_padding,
703         watermark=localized_watermark,
704     )
705     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
706
707     notify("stitching")
708     local_src = ""
709     try:
710         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
711         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
712         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
713         local_src = storage.acquire_local_input(
714             video["filepath"], getattr(cfg, "storage", None)
715         )
716         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
717     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
718         logger.error(
719             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
720         )
721         return {
722             "status": "failed",
723             "message": f"Highlight stitching failed: {e}",
724             "video_id": video_id,
725         }
726     finally:
727         if local_src:
728             storage.cleanup_acquired_local_input(
729                 video["filepath"], local_src, getattr(cfg, "storage", None)
730             )
731
732     # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
733     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
734     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
735     notify("finished")
736     return {
737         "status": "success",
738         "filepath": out_path,
739         "download_url": download_url,
740         "video_id": video_id,
741     }
742

```

8. user (2026-07-02T09:21:23.366Z)

tests/test_api_annotations.py:91: monkeypatch.setattr(m, "_remote_input_size_bytes", lambda

```

*a, **k: len(payload))
tests/test_api_assets.py:129: monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a,
**k: len(payload))
tests/test_api_cloud_dispatch.py:128: """Make backend.compute.get_compute_backend (re-
imported inside _maybe_dispatch_remote) build a
tests/test_api_cloud_dispatch.py:151: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:170: m._execute_processing(
tests/test_api_cloud_dispatch.py:196: m._execute_processing(
tests/test_api_cloud_dispatch.py:209: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:228: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:269: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:298: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:312: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:353: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:396: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:427: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:445: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:457: def test_capabilities_reports_cloud_available(cloud_env):
tests/test_api_cloud_dispatch.py:602: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:624: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:648: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:676: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:704: out =
m._execute_processing(m.ProcessRequest(video_path=str(video)))
tests/test_api_endpoints.py:815: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
len(payload))
tests/test_api_endpoints.py:833: assert m._report_config_for_input(local_cfg, str(tmp_path /
"local.mp4")) is local_cfg
tests/test_api_endpoints.py:839: assert m._report_config_for_input(explicit,
"gs://bucket/videos/x.mp4") is explicit
tests/test_api_endpoints.py:851: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10
* 1024**3)
tests/test_api_endpoints.py:868: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
None)
tests/test_async_jobs.py:159: """Item 5 positive: direct CLI-style path
(m._execute_processing) vs job-worker path
tests/test_async_jobs.py:173: direct_out = m._execute_processing(m.global_config,
m.db_instance, req)
tests/test_async_jobs.py:207: direct_f = m._execute_processing(m.global_config,
m.db_instance, req_f)
tests/test_async_jobs.py:226: de = m._execute_processing(m.global_config, m.db_instance,
req_e)
tests/test_async_jobs.py:489: def test_worker_dispatches_compile_kind_to_execute_compile(env):
tests/test_async_jobs.py:490: """The claimed-job dispatcher routes kind='compile' to
_execute_compile (NOT _execute_processing),
tests/test_async_jobs.py:503: mp.setattr(m, "_execute_compile", _fake_compile)
tests/test_async_jobs.py:506: "_execute_processing",
tests/test_async_jobs.py:507: lambda *a, **k: pytest.fail("a compile job must not hit
_execute_processing"),
tests/test_base.py:135: Exercise the exact duck-typing patterns used in main.py (both the
_execute_processing
tests/test_base.py:181: This allows the except blocks in main.py (_execute_processing:193-198
and the
tests/test_job_worker_module_seam.py:6:(`_arm_wake_timer`, `_get_executor`, `db_instance`,
`_execute_processing`)
Binary file tests/__pycache__/test_api_annotations.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_api_assets.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_api_endpoints.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_async_jobs.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_base.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_job_worker_module_seam.cpython-313-pytest-9.0.3.pyc matches

```

```

tests/test_api_annotations.py:24: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_api_cloud_dispatch.py:114: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_api_cloud_dispatch.py:382: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_api_cloud_dispatch.py:557: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_api_endpoints.py:32: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_api_endpoints.py:411: mp.setattr(m, "VideoCompiler", _FakeCompiler)
tests/test_api_endpoints.py:455: mp.setattr(m, "VideoCompiler", _CaptureCompiler)
tests/test_api_endpoints.py:486: mp.setattr(m, "VideoCompiler", _fake_ok_compiler())
tests/test_api_endpoints.py:531: mp.setattr(m, "VideoCompiler", _BoomCompiler)
tests/test_api_endpoints.py:575: mp.setattr(m, "VideoCompiler", _FakeCompiler)
tests/test_api_endpoints.py:782: mp.setattr(m, "emit_indexer_report", fake_emit)
tests/test_api_endpoints.py:817: mp.setattr(m, "compute_video_id", fail_hash)
tests/test_api_endpoints.py:1016: monkeypatch.setattr(m, "load_app_config", lambda *a, **k:
cfg)
tests/test_api_endpoints.py:1017: monkeypatch.setattr(m, "Database", lambda *a, **k: db)
tests/test_api_endpoints.py:1066: monkeypatch.setattr(m, "load_app_config", lambda *a, **k:
cfg)
tests/test_api_endpoints.py:1067: monkeypatch.setattr(m, "Database", lambda *a, **k: db)
tests/test_async_jobs.py:40: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_async_jobs.py:558: mp.setattr(m, "_get_executor", lambda: pool)
tests/test_edge_auth.py:233: monkeypatch.setattr(m, "_get_executor", lambda: _Noop())
tests/test_job_worker_loop.py:46: monkeypatch.setattr(m, "_get_executor", lambda:
_InlineExecutor())
tests/test_job_worker_loop.py:50: monkeypatch.setattr(m, "_arm_wake_timer", lambda delay, *a:
armed.append(delay))
tests/test_job_worker_module_seam.py:7: THROUGH the `backend.main` module object, so that
`monkeypatch.setattr(m, ...)`
tests/test_job_worker_module_seam.py:51: monkeypatch.setattr(m, "_arm_wake_timer", lambda
delay, *a: armed.append(delay))
tests/test_job_worker_module_seam.py:74: monkeypatch.setattr(m, "_get_executor", lambda:
_RecordingExecutor())
tests/test_main_module_budget.py:14: ``monkeypatch.setattr(backend.main, ...)` are load-
bearing seams ? see

```

10. user (2026-07-02T09:21:48.356Z)

(Bash completed with no output)

11. user (2026-07-02T09:22:03.833Z)

```

61:from backend.pipeline.stitcher.compiler import VideoCompiler
64:from backend.reporting import emit_indexer_report
74:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
359: # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
822: MD5 (`compute_video_id`, D3) so it joins the pipeline by content hash, and appears in
891: video_id = compute_video_id(local_working)
1437: compiler = VideoCompiler(
1559: video_id = compute_video_id(local_video)
1621: paths = emit_indexer_report(

```

12. user (2026-07-02T09:22:18.222Z)

```

380 assert r.status_code == 200
381 assert len(r.json()["play_segments"]) == 1
382
383
384 def test_compile_rejects_traversal_output_filename(env):
385     client, db, tmp_path, mp = env

```

```

386     db.add_video("vid", str(_video(tmp_path)), "processed", [])
387     sid = db.add_play_segment("vid", 1.0, 4.0)
388     r = client.post(
389         "/api/compile",
390         json={
391             "video_id": "vid",
392             "segment_ids": [sid],
393             "output_filename": "../evil.mp4",
394         },
395     )
396     assert r.status_code == 400
397
398
399     def test_compile_success_with_mocked_compiler(env):
400         client, db, tmp_path, mp = env
401         db.add_video("vid", str(_video(tmp_path)), "processed", [])
402         sid = db.add_play_segment("vid", 1.0, 4.0)
403
404         class _FakeCompiler:
405             def __init__(self, out_dir, watermark=None, **kwargs):
406                 pass
407
408             def compile_highlights(self, raw, pairs, name):
409                 return f"output/{name}"
410
411         mp.setattr(m, "VideoCompiler", _FakeCompiler)
412         # Async (#329): submit ? 202 + job_id; the inline executor runs the stitch; poll for
the result.
413         r = client.post(
414             "/api/compile",
415             json={"video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"},
416         )
417         assert r.status_code == 202
418         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
419         assert job["status"] == "done"
420         body = job["result"]
421         assert body["status"] == "success"
422         assert body["filepath"] == "output/out.mp4"
423         # the browser-reachable URL is returned so the SPA needn't hand-build it (P6)
424         assert body["download_url"] == "/api/highlights/vid/out.mp4"
425
426
427     def test_compile_resolves_bucket_source_to_local_before_stitching(env):
428         """A video ingested from a bucket URI stores its SOURCE ref (gs://?) as `filepath`.
/api/compile
429         must resolve that to a LOCAL file before stitching ? else ffmpeg can't open the
gs:// path and the
430         build 500s. Regression for the in-UI bucket?reel CUJ (B-P2). The local case stays
identity (covered
431         by test_compile_success_with_mocked_compiler)."""
432         client, db, tmp_path, mp = env
433         db.add_video(
434             "bvid", "gs://bucket/videos/x.mp4", "processed", []
435         ) # filepath = the SOURCE URI
436         sid = db.add_play_segment("bvid", 1.0, 4.0) # no shot_count ? exempt from the floor
437
438         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
439             with open(dst, "wb") as fh:
440                 fh.write(b"fetch-source-bytes")
441             return dst
442
443         mp.setattr(m.storage, "fetch", fake_fetch)
444
445         class _CaptureCompiler:
446             last_raw = None

```

```

447
448     def __init__(self, out_dir, watermark=None, **kwargs):
449         pass
450
451     def compile_highlights(self, raw, pairs, name):
452         _CaptureCompiler.last_raw = raw
453         return f"output/{name}"
454
455     mp.setattr(m, "VideoCompiler", _CaptureCompiler)
456     r = client.post(
457         "/api/compile",
458         json={"video_id": "bvid", "segment_ids": [sid], "output_filename": "reel.mp4"},
459     )
460     assert r.status_code == 202
461     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
462     assert job["status"] == "done" and job["result"]["status"] == "success"
463     # The stitcher received a FETCHED LOCAL path, NOT the gs:// URI.
464     assert _CaptureCompiler.last_raw is not None
465     assert not _CaptureCompiler.last_raw.startswith("gs://")
466     assert os.path.basename(_CaptureCompiler.last_raw) == "x.mp4"
467     assert not os.path.exists(_CaptureCompiler.last_raw)
468
469
470     def _fake_ok_compiler():
471         class _FakeCompiler:
472             def __init__(self, *a, **k):
473                 pass
474
475             def compile_highlights(self, raw, pairs, name):
476                 return f"output/{name}"
477
478         return _FakeCompiler
479
480
481     def test_compile_returns_202_with_poll_envelope(env):
482         """#329: /api/compile is async ? submit returns 202 {job_id, status:'queued', poll}
483         like /api/process."""
484         client, db, tmp_path, mp = env
485         db.add_video("vid", str(_video(tmp_path)), "processed", [])
486         sid = db.add_play_segment("vid", 1.0, 4.0)
487         mp.setattr(m, "VideoCompiler", _fake_ok_compiler())
488         r = client.post(
489             "/api/compile",
490             json={"video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"},
491         )
492         assert r.status_code == 202
493         body = r.json()
494         assert body["status"] == "queued"
495         assert body["poll"] == f"/api/jobs/{body['job_id']}"
496
497     def test_compile_unknown_video_404_at_submit(env):
498         """The cheap checks still fast-fail at SUBMIT (4xx), before any job is queued."""
499         client, db, tmp_path, mp = env
500         r = client.post(
501             "/api/compile",
502             json={"video_id": "nope", "segment_ids": [1], "output_filename": "out.mp4"},
503         )
504         assert r.status_code == 404
505
506
507     def test_compile_no_matching_segments_400_at_submit(env):
508         client, db, tmp_path, mp = env
509         db.add_video("vid", str(_video(tmp_path)), "processed", [])
510         r = client.post(

```

```

511         "/api/compile",
512         json={"video_id": "vid", "segment_ids": [999], "output_filename": "out.mp4"},
513     )
514     assert r.status_code == 400
515
516
517     def test_compile_stitch_failure_marks_job_done_with_failed_result(env):
518         """A stitch/ffmpeg error becomes a clean done+failed RESULT (not a worker crash /
519         'failed' status),
520         so the SPA always gets a verdict + message rather than a silent hang or an opaque
521         error."""
522         client, db, tmp_path, mp = env
523         db.add_video("vid", str(_video(tmp_path)), "processed", [])
524         sid = db.add_play_segment("vid", 1.0, 4.0)
525
526         class _BoomCompiler:
527             def __init__(self, *a, **k):
528                 pass
529
530             def compile_highlights(self, raw, pairs, name):
531                 raise RuntimeError("ffmpeg exploded")
532
533         mp.setattr(m, "VideoCompiler", _BoomCompiler)
534         r = client.post(
535             "/api/compile",
536             json={"video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"},
537         )
538         assert r.status_code == 202
539         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
540         assert job["status"] == "done" # the WORKER finished cleanly
541         assert job["result"]["status"] == "failed" # the pipeline VERDICT is a failure
542         assert "ffmpeg exploded" in job["result"]["message"]
543
544     def test_health_ok(env):
545         client = env[0]
546         r = client.get("/api/health")
547         assert r.status_code == 200
548         body = r.json()
549         assert body["status"] == "ok"
550         assert body["sport"] == "badminton"
551         assert "default_segmenter" in body # readiness fields present for the SPA banner
552
553     def test_compile_enforces_config_min_shot_count(env):
554         """stitching.min_shot_count applies on the API path (parity with
555         run_process_and_stitch.py): below-floor segments are dropped; segments
556         WITHOUT shot_count metadata are exempt (explicitly requested ids must not
557         silently vanish under the default floor)."""
558         client, db, tmp_path, mp = env
559         m.global_config.stitching.min_shot_count = 8
560         db.add_video("vid", str(_video(tmp_path)), "processed", [])
561         s_low = db.add_play_segment("vid", 1.0, 4.0, None, None, {"shot_count": 5})
562         s_high = db.add_play_segment("vid", 10.0, 14.0, None, None, {"shot_count": 12})
563         s_unknown = db.add_play_segment("vid", 20.0, 24.0) # no shot_count metadata
564
565         pairs_seen = []
566
567         class _FakeCompiler:
568             def __init__(self, out_dir, **kwargs):
569                 pass
570
571             def compile_highlights(self, raw, pairs, name):
572                 pairs_seen.extend(pairs)
573                 return f"output/{name}"

```

```

574
575     mp.setattr(m, "VideoCompiler", _FakeCompiler)
576     # Async (#329): the inline executor runs the stitch during the POST, so the floor
still applies.
577     r = client.post(
578         "/api/compile",
579         json={
580             "video_id": "vid",
581             "segment_ids": [s_low, s_high, s_unknown],
582             "output_filename": "reel.mp4",
583         },
584     )
585     assert r.status_code == 202
586     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
587     assert job["status"] == "done" and job["result"]["status"] == "success"
588     assert (10.0, 14.0) in pairs_seen and (20.0, 24.0) in pairs_seen
589     assert (1.0, 4.0) not in pairs_seen
590
591
592 def test_compile_400_when_all_below_min_shot_count(env):
593     client, db, tmp_path, mp = env
594     m.global_config.stitching.min_shot_count = 8
595     db.add_video("vid", str(_video(tmp_path)), "processed", [])
596     sid = db.add_play_segment("vid", 1.0, 4.0, None, None, {"shot_count": 3})
597     r = client.post(
598         "/api/compile",
599         json={"video_id": "vid", "segment_ids": [sid], "output_filename": "reel.mp4"},
600     )
601     assert r.status_code == 400
602     assert "min_shot_count" in r.json()["detail"]
603
604
605 def test_compile_uses_configured_stitching_paddings(env):
606     """config.json stitching.{begin,end}_padding must reach the ffmpeg trim boundaries.
607     Regression for the 2026-06-10 audit finding (d): /api/compile constructed
608     VideoCompiler without the paddings, so reels always used the constructor
609     defaults (0.5/1.0) and the config values were silently ignored."""

```

13. user (2026-07-02T09:22:32.172Z)

```

    doctor_path = os.path.join(
        os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
        "scripts",
        "doctor.py",
    )
    print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
    subprocess.run([sys.executable, doctor_path])
    return

if args.trim_start is not None and args.trim_end is not None:
    if not args.video_path:
        print("[ERROR] Please provide --video-path to extract a segment.")
        return

# Determine output path
out_filename = args.trim_output or "trimmed_segment.mp4"
if os.path.isabs(out_filename):
    out_path = out_filename
    out_dir = os.path.dirname(out_path)
    out_name = os.path.basename(out_path)
else:
    out_dir = resolve_output_dir(
        args.output_dir or "output"
    ) # P0a: app-home aware (CWD-identical when unset)
    out_path = os.path.join(out_dir, out_filename)

```

```

        out_name = out_filename

    os.makedirs(out_dir, exist_ok=True)
    print(
        f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s from
{args.video_path}..."
    )

    # global_config isn't initialized this early; load the typed config so the trim
    # clip carries the same configured stitching paddings + watermark as a real
    # highlight (watermark default-on).
    stitching = load_app_config().stitching
    compiler = VideoCompiler(
        out_dir,
        begin_padding=stitching.begin_padding,
        end_padding=stitching.end_padding,
        watermark=stitching.watermark,
    )
    try:
        res_path = compiler.compile_highlights(
            args.video_path, [(args.trim_start, args.trim_end)], out_name
        )
        print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
    except Exception as e:
        print(f"[ERROR] Failed to compile trimmed segment: {e}")
    return

if args.debug_clip is not None:
    if not args.video_path:
        print("[ERROR] Please provide --video-path to debug a specific clip.")
        return
    run_cli_diagnose_clip(args.video_path, args.debug_clip)
    return

# P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly `--app`
# launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
# no API key, no GPU, no cloud bills) at the resolved app-home (default_config_path(), which
# roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE load_app_config
reads
# it ? so a fresh `pip`/`uv` tool install` + `indexer --app` just works. NEVER overwrites an
# existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to --app
# (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
if args.app:
    seeded_path, seeded = write_light_default_config()
    if seeded:
        print(
            f"[APP] First run: seeded a truly-local default config at {seeded_path} "
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
60: is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
83: return _main.ProcessRequest(
88: # v8 compute-picker: the worker runs _execute_processing ? _maybe_dispatch_remote,
which
114:         result = _main._execute_compile(
121:         req = _main._job_to_request(job)
122:         result = _main._execute_processing(
138: except _main.JobWaiting as w:
229:     _main._run_claimed_job(job, config, db)
233: _main._schedule_next_drain_if_waiting(config, db)
251: _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
262:     lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),

```

```

755         rec = db.get_video(job["video_id"])
756         assert rec is not None and rec["filepath"] == "gs://bucket/videos/x.mp4"
757
758
759     def test_process_remote_input_reports_use_durable_output_dir(env):
760         """Remote input scratch is deleted after processing, so default report output must
761         be durable."""
762         client, db, tmp_path, mp = env
763         report_base = tmp_path / "out"
764         mp.setattr(m.global_config.output, "output_dir", str(report_base))
765         captured = []
766
767         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
768             with open(dst, "wb") as fh:
769                 fh.write(b"fetch-video-bytes")
770             return dst
771
772         def fake_emit(**kwargs):
773             captured.append(kwargs)
774             return {
775                 "json": os.path.join(
776                     kwargs["config"]["report"]["output_dir"], "report.json"
777                 )
778             }
779
780         mp.setattr(m.storage, "fetch", fake_fetch)
781         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
782         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
783         mp.setattr(m, "emit_indexer_report", fake_emit)
784         _CaptureSeg.last_path = None
785
786         r = client.post(
787             "/api/process",
788             json={"video_path": "gs://bucket/videos/x.mp4", "segmenter": "x"},
789         )
790         assert r.status_code == 202
791         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
792         assert job["status"] == "done"
793
794         assert captured
795         assert captured[0]["video_path"] == _CaptureSeg.last_path
796         assert captured[0]["config"]["report"]["output_dir"] == os.path.join(
797             str(report_base), "reports"
798         )
799         assert not os.path.exists(_CaptureSeg.last_path)
800
801     def test_process_remote_input_cleans_scratch_when_hash_fails(env):
802         client, db, tmp_path, mp = env
803         payload = b"fetch-video-bytes"
804         scratch_dirs = []
805
806         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
807             scratch_dirs.append(os.path.dirname(dst))
808             with open(dst, "wb") as fh:
809                 fh.write(payload)
810             return dst
811
812         def fail_hash(path):
813             raise RuntimeError("hash boom")
814
815         mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: len(payload))
816         mp.setattr(m.storage, "fetch", fake_fetch)
817         mp.setattr(m, "compute_video_id", fail_hash)
818         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)

```

```

819
820     r = client.post(
821         "/api/process",
822         json={"video_path": "gs://bucket/videos/x.mp4", "segmenter": "x"},
823     )
824     assert r.status_code == 202
825     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
826     assert job["status"] == "failed"
827     assert "hash boom" in job["error"]
828     assert scratch_dirs and all(not os.path.exists(d) for d in scratch_dirs)
829
830
831     def test_report_config_preserves_local_and_explicit_report_output(tmp_path):
832         local_cfg = AppConfig()
833         assert m._report_config_for_input(local_cfg, str(tmp_path / "local.mp4")) is
local_cfg
834
835         explicit = {
836             "output": {"output_dir": str(tmp_path / "out")},
837             "report": {"output_dir": str(tmp_path / "custom-reports")},
838         }
839         assert m._report_config_for_input(explicit, "gs://bucket/videos/x.mp4") is explicit
840
841
842     def test_process_remote_input_507_when_scratch_too_small(env):
843         """P2 (D11): a bucket source whose fetched copy won't fit in scratch fails fast with
507 at
844         submit ? before a job is queued ? rather than crashing the worker mid-fetch."""
845         import types
846
847         from backend.utils import freespace
848
849         client, db, tmp_path, mp = env
850         # Stat says the object is 10 GB; the scratch fs reports only 1 MB free ? shortfall.
851         mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10 * 1024**3)
852         mp.setattr(
853             freespace.shutil,
854             "disk_usage",
855             lambda p: types.SimpleNamespace(total=0, used=0, free=1024 * 1024),
856         )
857         r = client.post(
858             "/api/process",
859             json={"video_path": "gs://bucket/videos/big.mp4", "segmenter": "x"},
860         )
861         assert r.status_code == 507
862         assert "space" in r.json()["detail"].lower()
863
864
865     def test_process_remote_input_unstatable_is_not_blocked(env):
866         """Best-effort: a remote we can't size (stat returns None) must NOT be blocked by
the guard."""
867         client, db, tmp_path, mp = env
868         mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: None)
869
870         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
871             with open(dst, "wb") as fh:
872                 fh.write(b"bytes")
873             return dst
874
875         mp.setattr(m.storage, "fetch", fake_fetch)
876         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
877         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
878         r = client.post(
879             "/api/process",
880             json={"video_path": "gs://bucket/videos/x.mp4", "segmenter": "x"},

```

```
881         )
882         assert r.status_code == 202 # unmeasurable ? guard degrades to a no-op
883
884
```

15. assistant (2026-07-02T09:24:38.895Z)

Now let me scan test_api_cloud_dispatch.py's patching, and the paddings test, plus check for any other backend consumers of moved names.

16. user (2026-07-02T09:24:42.882Z)

```
102: monkeypatch.setattr(m, "db_instance", db)
103: monkeypatch.setattr(m, "db_instance", db)
112: monkeypatch.setattr(m, "global_config", cfg)
113: monkeypatch.setattr(m, "global_config", cfg)
114: monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
118: monkeypatch.setattr(
121: monkeypatch.setattr(
132: monkeypatch.setattr(
240: monkeypatch.setattr(m, "db_instance", db)
241: monkeypatch.setattr(m, "db_instance", db)
243: monkeypatch.setattr(m, "global_config", cfg)
244: monkeypatch.setattr(m, "global_config", cfg)
245: monkeypatch.setattr(m, "global_config", cfg)
246: monkeypatch.setattr(
268: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
351: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
372: monkeypatch.setattr(m, "db_instance", db)
381: monkeypatch.setattr(m, "global_config", cfg)
382: monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
385: monkeypatch.setattr(
388: monkeypatch.setattr(
414: monkeypatch.setattr(m, "db_instance", db)
418: monkeypatch.setattr(m, "global_config", cfg)
419: monkeypatch.setattr(
518: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
547: monkeypatch.setattr(m, "db_instance", db)
556: monkeypatch.setattr(m, "global_config", cfg)
557: monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
560: monkeypatch.setattr(
563: monkeypatch.setattr(
622: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
637: monkeypatch.setattr(m, "db_instance", db)
638: monkeypatch.setattr(m, "db_instance", db)
639: monkeypatch.setattr(m, "global_config", AppConfig())
641: monkeypatch.setattr(
644: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
658: monkeypatch.setattr(m, "db_instance", db)
659: monkeypatch.setattr(m, "db_instance", db)
660: monkeypatch.setattr(
668: monkeypatch.setattr(
```

17. user (2026-07-02T09:24:44.245Z)

```
def test_compile_uses_configured_stitching_paddings(env):
    """config.json stitching.{begin,end}_padding must reach the ffmpeg trim boundaries.
    Regression for the 2026-06-10 audit finding (d): /api/compile constructed
    VideoCompiler without the paddings, so reels always used the constructor
    defaults (0.5/1.0) and the config values were silently ignored."""
    client, db, tmp_path, mp = env
    cfg = m.global_config
    cfg.output.output_dir = str(tmp_path / "out")
    cfg.stitching.begin_padding = 2.0
```

```

cfg.stitching.end_padding = 3.0
cfg.stitching.watermark.enabled = False # keep the trim command minimal
db.add_video("vid", str(_video(tmp_path)), "processed", [])
sid = db.add_play_segment("vid", 10.0, 14.0)

calls = []

def fake_run(cmd, *a, **k):
    calls.append(cmd)
    if cmd[0] == "ffprobe":
        return MagicMock(stdout="100.0\n", returncode=0)
    out = cmd[-1]
    if isinstance(out, str) and out.endswith(".mp4"):
        open(out, "wb").write(b"clip")
    return MagicMock(returncode=0, stdout=b"", stderr=b"")

with patch.object(subprocess, "run", side_effect=fake_run):
    # The inline executor runs the stitch (under this patch) during the POST, so the ffmpeg
    # trim commands are captured here; then the job is already 'done'.
    r = client.post(
        "/api/compile",
        json={
            "video_id": "vid",
            "segment_ids": [sid],
            "output_filename": "reel.mp4",
        },
    )
    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
assert r.status_code == 202
assert job["status"] == "done" and job["result"]["status"] == "success"

trim = next(c for c in calls if c[0] == "ffmpeg" and "-ss" in c)
# 10.0 - begin_padding 2.0 and 14.0 + end_padding 3.0; the unwired constructor
# defaults (0.5/1.0) would have produced 9.5/15.0 here.
assert trim[trim.index("-ss") + 1] == "8.0"
assert trim[trim.index("-to") + 1] == "17.0"

```

```

def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
    """Item 5 positive: same ProcessRequest through job worker path and direct CLI-style path
    produce byte-identical DB rows, reports, final play_segments (modulo timestamps).
    Exercises Failure and empty-run branches.
    """
    client, db, tpath, mp = env
    vid_file = _video(tpath)
    vid = m.compute_video_id(str(vid_file))
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))

```

18. user (2026-07-02T09:24:46.513Z)

```

backend/api/job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
`_execute_processing`,
backend/api/job_worker.py:60: is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend/api/job_worker.py:88: # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
backend/api/job_worker.py:114: result = _main._execute_compile(
backend/api/job_worker.py:122: result = _main._execute_processing(

```

19. user (2026-07-02T09:25:03.745Z)

```

@pytest.fixture
def cloud_env(tmp_path, monkeypatch):
    """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the bucket
    read an

```

```

identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
db = Database(str(tmp_path / "t.db")) # noqa: F841
monkeypatch.setattr(m, "db_instance", db)
monkeypatch.setattr(m, "db_instance", db)
bucket = tmp_path / "bucket"
bucket.mkdir()
cfg = AppConfig.model_validate(
    { # noqa: F841
        "deployment": {"compute_target": "cloud_run"},
        "storage": {"backend": "local", "output_root": str(bucket)},
    }
)
monkeypatch.setattr(m, "global_config", cfg)
monkeypatch.setattr(m, "global_config", cfg)
monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
# A real gs:// input would be fetched to hash+validate; return a local fake so that path is
offline.
fake_local = tmp_path / "fetched.mp4"
fake_local.write_bytes(b"FAKEVIDEOBYTES")
monkeypatch.setattr(
    m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
)
monkeypatch.setattr(
    m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
)
return db, bucket, monkeypatch

def _inject_fake(monkeypatch, fake):
    """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote)
    build a
    real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token.
    """
    forwards the per-request ``target_override`` (item 3) so the cloud-picker path resolves
    too."""
    real = compute_pkg.get_compute_backend
    monkeypatch.setattr(
        compute_pkg,
        "get_compute_backend",
        lambda config, **kw: real(
            config, run_client=fake, policy=_FAST, token="tok", **kw
        ),
    )

    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "db_instance", db)
    cfg = AppConfig() # compute_target defaults to "" # noqa: F841
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    video = tmp_path / "v.mp4"
    video.write_bytes(b"hashable-bytes")

    class _InProcSeg:
        def __init__(self, db_, cfg_):
            self.db = db_

        def process_video(self, vid, path, *a, **k):
            sid = self.db.add_play_segment(vid, 0.0, 5.0)
            return [

```

```

        {
            "id": sid,
            "start_time": 0.0,
            "end_time": 5.0,
            "duration": 5.0,
            "metadata": {},
        }
    ]

monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
out = m._execute_processing(
    m.ProcessRequest(video_path=str(video), segmenter="motion")
fake_local = tmp_path / "fetched.mp4"
fake_local.write_bytes(b"FAKEVIDEOBYTES")
monkeypatch.setattr(
    m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
)
monkeypatch.setattr(
    m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
)
monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
) # no bucket, no env # noqa: F841
monkeypatch.setattr(m, "global_config", cfg)
monkeypatch.setattr(
    m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
)
monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
fake_local = tmp_path / "fetched.mp4"
fake_local.write_bytes(b"FAKEVIDEOBYTES")
monkeypatch.setattr(
    m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
)
monkeypatch.setattr(
    m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
)
monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
monkeypatch.setattr(m, "global_config", AppConfig())
m.app.state.config = AppConfig()
monkeypatch.setattr(
    m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
)
monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
video = tmp_path / "v.mp4"
video.write_bytes(b"hashable-bytes")
requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
db = Database(str(tmp_path / "t.db")) # noqa: F841
monkeypatch.setattr(m, "db_instance", db)
monkeypatch.setattr(m, "db_instance", db)
monkeypatch.setattr(
    m,
    "global_config",
    AppConfig.model_validate({"deployment": {"compute_target": ""}}),
)
m.app.state.config = AppConfig.model_validate(
    {"deployment": {"compute_target": ""}}
)
monkeypatch.setattr(
    m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
)
monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)

```

One final systematic check for other bare-consumed names patched directly on `m`.

21. user (2026-07-02T09:25:31.886Z)

```
---
tests/test_locale_analytics.py:12:from backend.main import ProcessRequest
---
```

22. user (2026-07-02T09:26:01.646Z)

```
..... [100%]
===== warnings summary =====
tests/test_api_endpoints.py::test_process_remote_input_reports_use_durable_output_dir
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
5 passed, 47 deselected, 1 warning in 6.72s
```

23. assistant (2026-07-02T09:26:19.909Z)

Surprising ? they pass. Let me check what `m` actually is in these tests and how the patch reaches orchestration.

24. user (2026-07-02T09:26:23.422Z)

```
"""TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
that broke `import backend.main` shipped green because nothing imported it)."""
```

```
import os
import subprocess
import sys
from unittest.mock import MagicMock, patch

import pytest

pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.pipeline.validators.base import ValidationResult

class _InlineExecutor:
    """submit() runs the fn synchronously ? async jobs complete before the response
    is inspected (full lifecycle coverage lives in test_async_jobs.py)."""

    def submit(self, fn, *args, **kwargs):
        fn(*args, **kwargs)

@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
    app = m.create_app(cfg, db)
    client = TestClient(app)
```

```
def _video(tmp_path, name="m.mp4"):
    p = tmp_path / name
    p.write_bytes(b"not-a-real-video-but-hashable")
```

Wait ? I need to verify which `backend` package pytest actually imported (worktree vs installed/main repo).

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\backend\orchestration.py
tests/test_api_endpoints.py::test_compile_success_with_mocked_compiler
```

```
exec_compile module: backend.orchestration
```

```
VideoCompiler line: ['compiler = _main.VideoCompiler(']
```

```
-rw-r--r-- 1 avidu 197609 76822 2026-07-02 14:45:27.283848900 +0530 backend/main.py
-rw-r--r-- 1 avidu 197609 34320 2026-07-02 14:54:54.480832200 +0530 backend/orchestration.py
362:         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
365:         elif req is None and isinstance(config, _main.ProcessRequest):
370:             config = _main.global_config
372:             db = _main.db_instance
386:             video_id = _main.compute_video_id(local_video)
571:             paths = _main.emit_indexer_report(
702:         compiler = _main.VideoCompiler(
21:import csv
22:import json
23:import logging
24:import math
25:import os
26:import tempfile
27:from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
28:from urllib.parse import quote
30:from backend import storage
```

```

31:from backend.config import AppConfig, StitchingConfig
32:from backend.pipeline.segmenters.base import segmenter_registry
33:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
34:from backend.pipeline.validators.base import registry
35:from backend.results import Failure
36:from backend.utils.paths import output_base, safe_output_filename

```

29. assistant (2026-07-02T09:28:03.109Z)

The file was updated at 14:54 (after my initial read) ? the author routed `compute\_video\_id`, `emit\_indexer\_report`, `VideoCompiler` through `\_main` at call time. Let me re-verify the current state fully and run the seam-heavy test files.

30. user (2026-07-02T09:28:06.874Z)

```

)

def _execute_processing(
    config=None, db=None, req=None, progress=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

    Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
    module-level ``global_config``/``db_instance`` (backward-compatible with existing
    tests that pass only ``req``). Route handlers always pass explicit params; older
    test code that calls ``_execute_processing(req)`` still works.

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main # call-time seam: main's globals/model stay patchable

    # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
    # shift positional args ? req arrives in position 0 (config), progress in position 1 (db).
    if config is not None and not isinstance(config, AppConfig):
        (response or {}).get("status") if isinstance(response, dict) else None,
    )
    # Always emit the per-video observability report (next to the video, or to
    # report.output_dir). Never raises. Surface the paths in the response. (The compute-path
    # proof lives on the job result + the [COMPUTE] log, not the report ? obsreport's
record_run
    # doesn't surface arbitrary run signals.)
    paths = _main.emit_indexer_report(
        db=db,
        video_id=video_id,
        # Resolved LOCAL path, not the raw URI: obsreport writes the report next to the
        # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://, local://)
        # collapses to a literal scheme dir at the repo root (see #397). The source URI is
        # still preserved in DB provenance via db.add_video.
        video_path=local_video,
        config=_report_config_for_input(config, req.video_path),
        val_results=val_results,
    )
    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel,
    and return the {status, filepath, download_url} result the SPA polls for. Returns a
    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a stitch
error)
    so the SPA always gets a clean verdict instead of a worker-crash error."""
    import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main

    notify = progress or (lambda stage: None)
    params = json.loads(job.get("params") or "{}")
    video_id = job.get("video_id") or ""

```

```

segment_ids = params.get("segment_ids") or []
locale = params.get("locale")

notify("resolving")
try:
    video, kept, out_name = _resolve_compile(
        db, cfg, video_id, segment_ids, params.get("output_filename", "")
    )
except _CompileError as e:
    return {"status": "failed", "message": e.detail, "video_id": video_id}

stitching = getattr(cfg, "stitching", None) or StitchingConfig()
output_dir = (
    getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
)
# Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
job);
# unknown/absent locale keeps the configured default unchanged.
localized_watermark = localize_watermark(stitching.watermark, locale)
compiler = _main.VideoCompiler(
    output_dir,
    begin_padding=stitching.begin_padding,
    end_padding=stitching.end_padding,
    watermark=localized_watermark,
)
time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

notify("stitching")
11:the ``backend.main`` module object at call time (``import backend.main as _main``), never as
a
356:     import backend.main as _main # call-time seam: main's globals/model stay patchable
679:     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main

```

31. user (2026-07-02T09:29:09.768Z)

```

tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
162 passed, 1 skipped, 8 warnings in 47.17s

```

32. user (2026-07-02T09:29:53.057Z)

```

46:def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
59:def _report_config_for_input(config: Any, raw_input: str) -> Any:
137:def _cloud_available(cfg) -> bool:
156:def _maybe_dispatch_remote(
216:     if not _cloud_available(cfg):
299:         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
311:     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
498:     remote = _maybe_dispatch_remote(
535:         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
579:         config=_report_config_for_input(config, req.video_path),
625:def _resolve_compile(
689:     video, kept, out_name = _resolve_compile(
692:     except _CompileError as e:
===

```

```
backend/orchestration.py:744
backend/main.py:1703
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
  backend/main.py | 757 ++++-----
1 file changed, 50 insertions(+), 707 deletions(-)
```

33. user (2026-07-02T09:30:45.662Z)

Structured output provided successfully